

Ghid complet de parcurgere a programei

Concursul național de ocupare a posturilor didactice/catedrelor vacante/ rezervate

Informatică și Tehnologia Informației

Acest material este construit după fișierul oficial 19_Programa_titu_P_Informatica_si_tehnologia_informatiei.pdf și are rolul de fișă centrală de studiu. Nu înlocuiește manualele, programele școlare și bibliografia, ci organizează conținuturile astfel încât pregătirea să fie completă, verificabilă și ușor de urmărit.

Cuprins

1. Cum se folosește ghidul	4
1.1 Obiectiv	4
1.2 Marcaje importante	4
1.3 Metodă recomandată de lucru	4
2. Competențe ale cadrului didactic de specialitate	4
2.1 Conținut științific	4
2.2 Competențe didactice	4
2.3 Checklist general	5
3. Tematica științifică	5
3.1 Algoritmi	5
3.1.1 Noțiunea de algoritm	5
3.1.2 Programarea structurată	5
3.1.3 Reprezentarea algoritmilor	5
3.1.4 Algoritmi elementari	6
3.1.5 Sortare	6
3.1.6 Interclasare	7
3.1.7 Căutare	7
3.1.8 Analiza complexității	7
3.2 Limbaje de programare: C, C++ și Pascal	7
3.2.1 Sintaxă și medii de programare	7
3.2.2 Elemente de bază	8
3.2.3 Tipuri structurate	8
3.2.4 Fișiere text	9
3.2.5 Subprograme	9
3.2.6 Recursivitate	10
3.2.7 Programarea orientată pe obiecte	10
3.3 Metode de programare	11
3.3.1 Greedy	11
3.3.2 Backtracking	11
3.3.3 Divide et impera	12
3.3.4 Programare dinamică	12

3.3.5 Generări combinatoriale	12
3.4 Alocarea dinamică a memoriei	13
3.4.1 Concept	13
3.4.2 Structuri liniare	13
3.4.3 Structuri arborescente	13
3.5 Teoria grafurilor	14
3.5.1 Definiții și reprezentări	14
3.5.2 Tipuri speciale	15
3.5.3 Parcurgeri	15
3.5.4 Conexitate și tare conexitate	15
3.5.5 Grafuri ponderate	16
3.5.6 Hamiltonian, eulerian și arbori	16
3.6 Baze de date	17
3.6.1 Concepte de bază	17
3.6.2 Model conceptual	17
3.6.3 Model relațional/fizic	17
3.6.4 Normalizare	18
3.6.5 SQL	18
3.7 Arhitectura generală a sistemelor de calcul	19
3.7.1 Sistem de calcul	19
3.7.2 Calculator personal	19
3.7.3 Ergonomie și siguranță	20
3.8 Sisteme de operare	20
3.8.1 Concepte	20
3.8.2 Interfață și accesorii	21
3.8.3 Sistem de fișiere	21
3.8.4 Securitate, viruși, licențe	21
3.9 Aplicații de birotică	22
3.9.1 Procesoare de text	22
3.9.2 Calcul tabelar	22
3.9.3 Prezentări	22
3.10 Rețele de calculatoare	23
3.10.1 Concepte	23
3.10.2 Internet	23
3.10.3 HTML	24
4. Metodica predării disciplinelor informatice	24
4.1 Proiectarea, organizarea și desfășurarea activității didactice	24
4.1.1 Curriculum național	24
4.1.2 Documente de proiectare	25
4.1.3 Demersuri didactice	25
4.2 Strategii didactice	26
4.2.1 Tipuri de strategii	26
4.2.2 Lecția	26
4.2.3 Metode didactice	26
4.2.4 Forme de organizare	27
4.2.5 Mijloace de învățământ și resurse digitale	27
4.3 Evaluarea randamentului școlar	28

4.3.1 Concept și funcții	28
4.3.2 Metode și instrumente	28
4.3.3 Itemi	28
4.3.4 Notare și erori de evaluare	29
5. Proba practică	30
5.1 Domenii de pregătit	30
5.2 Programare	30
5.3 Baze de date	30
5.4 Sisteme de operare și fișiere	30
5.5 Birotică	31
5.6 HTML și Internet	31
6. Plan de studiu	31
6.1 Etapa 1: Fundamentare algoritmică	31
6.2 Etapa 2: Metode avansate și structuri	32
6.3 Etapa 3: Grafuri	32
6.4 Etapa 4: Baze de date și TIC	32
6.5 Etapa 5: Metodică și evaluare	33
6.6 Etapa 6: Recapitulare pe subiecte	33
7. Matrice de verificare finală	33
7.1 Algoritmică și programare	33
7.2 Metode și structuri	34
7.3 Grafuri	34
7.4 Baze de date și TIC	34
7.5 Metodică	34
7.6 Matrice explicită de acoperire a programei	35
8. Bibliografie orientativă după programă	35
8.1 Tematica științifică	35
8.2 Metodica predării	36
9. Reguli pentru răspunsuri de punctaj maxim	36
9.1 La probleme de programare	36
9.2 La baze de date	36
9.3 La metodică	37
10. Legătura cu materialele din repository	37
11. Comandă de compilare	37

1. Cum se folosește ghidul

1.1 Obiectiv

Scopul este parcurgerea integrală a programei pentru Titularizare la Informatică și Tehnologia Informației, cu accent pe:

- cunoașterea conținuturilor științifice;
- capacitatea de a rezolva probleme de algoritmică și programare;
- proiectarea corectă a bazelor de date și formularea interogărilor SQL;
- stăpânirea noțiunilor TIC cerute la proba scrisă și proba practică;
- elaborarea răspunsurilor de metodică, proiectare didactică și evaluare.

1.2 Marcaje importante

Programa marchează cu (*) zonele care intră și la proba practică. În acest ghid ele sunt evidențiate prin eticheta **Practic**. Zonele respective sunt:

- capitolul 2: limbaje de programare, pentru elementele marcate în programă;
- capitolul 6: baze de date, pentru operațiile relaționale și SQL;
- capitolul 8: sisteme de operare, pentru operațiile și aplicațiile marcate;
- capitolul 9: aplicații de birotică;
- capitolul 10: rețele, Internet și HTML.

1.3 Metodă recomandată de lucru

Pentru fiecare capitol:

1. Citește sinteza teoretică.
2. Bifează checklistul de competențe.
3. Rezolvă minimum două exerciții reprezentative.
4. Caută în rezolvările din repository un subiect asemănător.
5. Scrie un răspuns de examen în limbaj complet, nu doar idei scurte.

2. Competențe ale cadrului didactic de specialitate

2.1 Conținut științific

Candidatul trebuie să cunoască informatica și TIC la nivelul disciplinelor din gimnaziu și liceu, pentru toate specializările relevante. Asta înseamnă nu doar definiții, ci și capacitatea de a explica, exemplifica, implementa și adapta conținutul pentru elevi.

De stăpânit:

- algoritmi elementari și avansați;
- limbaje de programare și medii de lucru;
- baze de date;
- sisteme de calcul, sisteme de operare, rețele;
- aplicații de birotică și instrumente digitale;
- concepte de didactică, proiectare și evaluare.

2.2 Competențe didactice

Profesorul trebuie să poată transforma conținutul informatic într-o situație de învățare. Răspunsurile de examen trebuie să arate:

- alegerea obiectivelor/competențelor;
- alegerea metodelor potrivite;

- organizarea activității pe momente;
- diferențierea sarcinilor;
- utilizarea mijloacelor tradiționale și digitale;
- evaluarea prin instrumente valide.

2.3 Checklist general

- Pot explica fiecare conținut științific din programă prin definiție, exemplu și aplicație.
- Pot scrie algoritmi în pseudocod, C++ și Pascal pentru cerințele uzuale.
- Pot analiza complexitatea timp/spațiu.
- Pot proiecta o bază de date până la model relațional și interogări SQL.
- Pot construi un proiect de lecție sau de unitate de învățare.
- Pot formula itemi, bareme, criterii de evaluare și metode de feedback.

3. Tematica științifică

3.1 Algoritmi

3.1.1 Noțiunea de algoritm

Un algoritm este o succesiune finită, clară și efectivă de pași prin care, pornind de la date de intrare, se obțin date de ieșire. Caracteristicile esențiale sunt:

- finitudine;
- claritate/neambiguitate;
- generalitate;
- determinism, acolo unde problema o cere;
- efectivitate;
- existența datelor de intrare și de ieșire.

La examen: La examen se cere frecvent diferența dintre algoritm și program: algoritmul este soluția abstractă, iar programul este implementarea într-un limbaj de programare.

3.1.2 Programarea structurată

Programarea structurată se bazează pe trei structuri fundamentale:

- secvența;
- decizia;
- repetiția.

Principiile importante:

- descompunerea problemei în pași simpli;
- evitarea salturilor necontrolate;
- folosirea subprogramelor;
- lizibilitate și verificabilitate;
- date bine delimitate.

3.1.3 Reprezentarea algoritmilor

Trebuie stăpânite cel puțin trei reprezentări:

- limbaj natural structurat;
- pseudocod;
- scheme logice sau blocuri grafice.

Pentru medii vizuale interactive, ideea centrală este reprezentarea instrucțiunilor prin blocuri grafice care pot fi compuse prin drag-and-drop. Concepte cheie:

- **Evenimente:** blocuri de start („când se dă clic pe steag”, „când se apasă tasta”);
- **Structuri de control:** bucle repetitive („repetă de 10 ori”, „la nesfârșit”, „repetă până când”) și decizionale („dacă... atunci... altfel”);
- **Variabile și Liste:** declararea globală/locală a variabilelor și stocarea dinamică a datelor în liste;
- **Subprograme (Blocuri proprii):** definirea de noi blocuri cu parametri de intrare pentru modularizare.

Medii reprezentative: Scratch (educațional, bazat pe costume, sunete și coordonate), Blockly (bibliotecă Google pentru generare de cod), App Inventor (pentru aplicații mobile).

3.1.4 Algoritmi elementari

Teme obligatorii:

- prelucrarea cifrelor unui număr;
- divizibilitate;
- primalitate;
- cmmdc și cmmmc;
- prelucrarea secvențelor;
- minime, maxime, sume, numărări;
- șiruri recurente;
- frecvențe;
- interschimbări și transformări elementare.

Model de răspuns pentru prelucrarea cifrelor:

```
int suma_cifre(int n) {  
    int s = 0;  
    while (n > 0) {  
        s += n % 10;  
        n /= 10;  
    }  
    return s;  
}
```

Pentru numărul 0, cazul trebuie tratat explicit dacă problema cere numărul de cifre sau suma cifrelor.

3.1.5 Sortare

Algoritmii ceruți explicit:

- sortarea prin metoda bulelor;
- sortarea prin inserție;
- sortarea prin selecție;
- sortarea prin numărare.

Comparație esențială:

- bule, inserție, selecție: în general $O(n^2)$;
- numărare: $O(n + k)$, unde k este dimensiunea domeniului valorilor;
- sortarea prin inserție este bună pentru date aproape sortate;
- sortarea prin numărare cere domeniu finit și rezonabil.

3.1.6 Interclasare

Interclasarea combină două șiruri ordonate într-un singur șir ordonat, folosind doi indici. Complexitatea este $O(n + m)$.

```
int i = 0, j = 0;
while (i < n && j < m) {
    if (a[i] <= b[j]) c[k++] = a[i++];
    else c[k++] = b[j++];
}
while (i < n) c[k++] = a[i++];
while (j < m) c[k++] = b[j++];
```

3.1.7 Căutare

Căutarea secvențială:

- funcționează pe date nesortate;
- are complexitate $O(n)$.

Căutarea binară:

- cere date sortate;
- are complexitate $O(\log n)$;
- înjumătățește intervalul de căutare la fiecare pas.

3.1.8 Analiza complexității

Trebuie analizate:

- durata de executare;
- spațiul de memorie utilizat.

Pentru examen, răspunsul bun precizează:

- dimensiunea intrării;
- operația dominantă;
- ordinul de complexitate;
- eventual diferența între caz favorabil, mediu și defavorabil.

Checklist:

- Pot explica noțiunea de algoritm și caracteristicile sale.
- Pot reprezenta un algoritm în pseudocod și prin schemă logică.
- Pot implementa algoritmi elementari pe numere și secvențe.
- Pot compara sortările cerute în programă.
- Pot aplica interclasarea și căutarea binară.
- Pot calcula complexitatea pentru un algoritm simplu.

3.2 Limbaje de programare: C, C++ și Pascal

Practic: Elementele marcate cu (*) în programă intră și la proba practică.

3.2.1 Sintaxă și medii de programare

Un limbaj de programare are reguli lexicale, sintactice și semantice. La examen trebuie cunoscute:

- structura unui program;
- declararea variabilelor;
- tipurile de date;
- operatorii;

- instrucțiunile;
- subprogramele;
- compilarea și execuția.

Medii posibile:

- Code::Blocks, Dev-C++, Visual Studio Code pentru C/C++;
- Free Pascal/Lazarus pentru Pascal;
- medii online sau medii educaționale, unde sunt permise.

3.2.2 Elemente de bază

Concepte:

- vocabularul limbajului;
- identificatori;
- constante;
- variabile;
- tipuri simple;
- expresii;
- operatori;
- citire și scriere;
- comentarii;
- instrucțiuni simple și compuse.

Model C++:

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;
    cout << n * n << '\n';
    return 0;
}
```

Model Pascal:

```
program Exemplu;
var
    n: integer;
begin
    readln(n);
    writeln(n * n);
end.
```

3.2.3 Tipuri structurate

Teme:

- tablouri unidimensionale;
- matrice;
- înregistrări/structuri;
- șiruri de caractere;
- prelucrări specifice.

Pentru tablouri:

- parcurgere;
- căutare;
- sortare;
- inserare/ștergere;
- frecvențe;
- subsecvențe.

Pentru șiruri:

- lungime;
- acces la caractere;
- căutare de subșir;
- transformări;
- separarea cuvintelor;
- testarea proprietăților.

3.2.4 Fișiere text

Operații:

- deschidere;
- citire;
- scriere;
- închidere;
- verificarea sfârșitului de fișier.

C++:

```
#include <fstream>
using namespace std;

ifstream fin("date.in");
ofstream fout("date.out");
```

Pascal:

```
var f, g: text;
begin
    assign(f, 'date.in'); reset(f);
    assign(g, 'date.out'); rewrite(g);
    close(f); close(g);
end.
```

3.2.5 Subprograme

Subprogramele permit modularizarea rezolvării. Se cer:

- declarare;
- definire;
- apel;
- parametri transmiși prin valoare;
- parametri transmiși prin referință/variabilă;
- variabile locale și globale;
- domeniu de vizibilitate.

Subprograme predefinite:

- C/C++:

- <cstring>: strlen (lungime), strcpy/strncpy (copiere), strcat/strncat (concatenare), strcmp (comparare), strchr (căutare caracter), strstr (căutare subșir), strtok (separare în cuvinte).
- <cmath>: abs (modul), sqrt (rădăcină), pow (putere), floor (rotunjire în jos), ceil (rotunjire în sus), sin, cos.
- <cctype>: isalpha, isdigit, isalnum, islower, isupper, tolower, toupper.
- **Pascal:**
 - String-uri: length (lungime), concat (concatenare), copy (extragere subșir), pos (poziție subșir), delete (ștergere caractere), insert (inserare text), val (conversie text-număr), str (conversie număr-text).
 - Matematici: abs (modul), sqr (pătrat), sqrt (rădăcină), round (rotunjire), trunc (trunchiere), sin, cos.

Regulă de examen: folosește subprogram când problema are o prelucrare repetată sau când cerința o solicită explicit.

3.2.6 Recursivitate

Un subprogram recursiv se apelează pe sine. Orice rezolvare recursivă corectă trebuie să aibă:

- caz de bază;
- reducerea problemei;
- apel recursiv;
- combinarea rezultatelor, dacă este necesar.

Exemplu:

```
int suma(int n) {  
    if (n == 0) return 0;  
    return n % 10 + suma(n / 10);  
}
```

3.2.7 Programarea orientată pe obiecte

Noțiuni:

- abstractizarea datelor;
- încapsulare;
- moștenire;
- polimorfism;
- clasă și obiect;
- constructor și destructor;
- niveluri de acces;
- supraîncărcare;
- derivarea claselor.

Model minim C++:

```
class Elev {  
private:  
    string nume;  
    double media;  
public:  
    Elev(string n, double m) : nume(n), media(m) {}  
    double getMedia() const { return media; }  
};
```

Checklist:

- Pot scrie programe simple în C++ și Pascal.
- Pot utiliza tablouri, matrice, structuri și șiruri.
- Pot lucra cu fișiere text.
- Pot proiecta și apela subprograme.
- Pot explica recursivitatea cu stivă de apeluri.
- Pot explica principiile POO și da exemple.

3.3 Metode de programare

3.3.1 Greedy

Metoda greedy construiește soluția pas cu pas, alegând la fiecare etapă opțiunea local optimă. Este corectă doar când problema are proprietatea alegerii greedy și substructură optimă.

Exemple:

- rucsacul în varianta continuă;
- problema spectacolelor;
- selectarea intervalelor compatibile;
- unele probleme de optimizare pe grafuri, cum ar fi Kruskal și Prim.

Răspuns complet:

- descrii strategia;
- justifici alegerea locală;
- arăți că soluția este fezabilă;
- precizezi complexitatea.

3.3.2 Backtracking

Backtracking generează soluții candidate și abandonează ramurile care nu mai pot duce la o soluție validă.

Elemente:

- vector de soluție;
- nivel;
- mulțime de candidați;
- condiții de validare;
- condiție de soluție;
- revenire.

Exemple:

- problema celor n regine;
- labirint;
- comis-voiajor;
- generarea permutărilor, aranjamentelor, combinărilor.

Schelet:

```
void back(int k) {  
    if (esteSolutie(k)) afiseaza();  
    else {  
        for (int x : candidati(k)) {  
            sol[k] = x;  
            if (valid(k)) back(k + 1);  
        }  
    }  
}
```

```
    }  
  }  
}
```

3.3.3 Divide et impera

Strategie:

1. împarte problema în subprobleme;
2. rezolvă subproblemele;
3. combină rezultatele.

Exemple:

- turnurile din Hanoi;
- sortarea prin interclasare;
- sortarea rapidă;
- căutarea binară;
- determinarea maximului prin împărțire.

3.3.4 Programare dinamică

Programarea dinamică rezolvă probleme cu subprobleme suprapuse și substructură optimă. Se folosește memorarea rezultatelor sau completarea unui tabel.

Exemple:

- cel mai lung subșir crescător;
- rucsacul discret;
- cel mai lung subșir comun;
- drumuri minime în anumite contexte;
- probleme de numărare.

Pași:

- definește starea;
- definește relația de recurență;
- stabilește cazurile inițiale;
- precizează ordinea de calcul;
- extrage rezultatul;
- analizează complexitatea.

3.3.5 Generări combinatoriale

Trebuie cunoscute:

- permutări;
- aranjamente;
- combinări;
- submulțimi;
- partițiile unui număr natural;
- partițiile unei mulțimi.

Pentru fiecare, trebuie știut:

- ce se generează;
- câte soluții există;
- cum se evită duplicatele;
- ce condiții de validare se aplică.

Checklist:

- Pot identifica metoda potrivită pentru o problemă.
- Pot scrie scheletul pentru greedy, backtracking, divide et impera și programare dinamică.
- Pot justifica eficiența și corectitudinea la nivel de examen.
- Pot genera elemente combinatoriale fără duplicate.

3.4 Alocarea dinamică a memoriei

3.4.1 Concept

Alocarea dinamică permite rezervarea memoriei în timpul execuției, nu doar la compilare. Este utilă când dimensiunea datelor nu este cunoscută de la început sau când structura crește/scade.

În C++:

- `new` alocă;
- `delete` eliberează;
- pointerii rețin adrese.

3.4.2 Structuri liniare

Liste simplu înlănțuite:

- fiecare nod are informație și legătură spre următorul;
- operații: creare, inserare, ștergere, parcurgere, căutare.

Liste dublu înlănțuite:

- fiecare nod are legături spre precedent și următor;
- permit deplasare în ambele sensuri.

Liste circulare:

- ultimul nod indică primul nod;
- utile în probleme cu rotații sau procese ciclice.

Stive:

- LIFO;
- operații push, pop, top.

Cozi:

- FIFO;
- operații enqueue, dequeue, acces la început.

3.4.3 Structuri arborescente

Arbori cu rădăcină:

- nod rădăcină;
- relație părinte-copil;
- nivel, înălțime, frunze.

Arbori binari:

- fiecare nod are cel mult doi fii;
- parcurgeri: RSD (preordine), SRD (inordine), SDR (postordine).

Arbori binari speciali:

- **Arbore binar complet:** arbore binar în care toate nivelurile sunt complet ocupate, cu excepția eventuală a ultimului nivel care este completat de la stânga la dreapta.

- **Arbore binar de căutare (ABC):** în stânga valori mai mici decât rădăcina, în dreapta valori mai mari; parcurgerea SRD (inordine) a unui ABC generează valorile în ordine strict crescătoare.
- **Heap:** un arbore binar complet care satisface proprietatea de Heap:
 - **Max-Heap:** valoarea oricărui nod este mai mare sau egală cu valorile fiilor săi ($A[\text{parent}(i)] \geq A[i]$).
 - **Min-Heap:** valoarea oricărui nod este mai mică sau egală cu valorile fiilor săi ($A[\text{parent}(i)] \leq A[i]$).

Reprezentarea arborilor:

- **Prin pointeri:** fiecare nod conține informația propriu-zisă și legături (adrese) către fii (stg și dr pentru arbori binari).
- **Prin vectori (pentru arbori compleți/heaps):** datele se stochează compact într-un tablou $A[1..N]$ fără pointeri:
 - Rădăcina este la indexul 1.
 - Fiul stâng al nodului i se află la indexul $2i$.
 - Fiul drept al nodului i se află la indexul $2i + 1$.
 - Părintele nodului i se află la indexul $\text{floor}(i / 2)$.

Operații specifice pe Heap:

- **Heapify** (cernere): corectează o singură încălcare a proprietății de heap pe un subarbore. Timp: $O(\log N)$.
- **Build-Heap:** transformă un tablou oarecare într-un heap, apelând Heapify de la $\lfloor \frac{N}{2} \rfloor$ în jos spre 1. Timp: $O(N)$.
- **Insert:** plasează elementul pe ultima poziție și îl deplasează în sus (promovare/sift-up) până se restabilește ordinea. Timp: $O(\log N)$.
- **Extract-Max / Extract-Min:** elimină rădăcina, o înlocuiește cu ultimul element din heap, reduce dimensiunea și apelează Heapify pe rădăcină. Timp: $O(\log N)$.
- **Heap-Sort:** algoritm de sortare în timp $O(N \log N)$ obținut prin extrageri succesive din heap.

Checklist:

- Pot explica pointerii și alocarea/eliberarea memoriei.
- Pot descrie liste, stive și cozi implementate dinamic.
- Pot descrie arborii și parcurgerile lor.
- Pot preciza operații specifice și complexități orientative.

3.5 Teoria grafurilor

3.5.1 Definiții și reprezentări

Graf neorientat: perechi de vârfuri fără orientare. Graf orientat: arce orientate de la un vârf la altul.

Metode de reprezentare:

- matrice de adiacență;
- liste de adiacență;
- matrice de incidență;
- listă de muchii/arce.

Terminologie fundamentală:

- **Grad**: gradul unui vârf v în graf neorientat (numărul de muchii incidente cu v). În graf orientat avem grad exterior $d^+(v)$ (arce care ies) și grad interior $d^-(v)$ (arce care intră).
- **Adiacență**: relație între două vârfuri unite de o muchie/arc.
- **Incidență**: relație între o muchie/arc și vârfurile sale de capăt.
- **Lanț** (în graf neorientat): o succesiune de vârfuri $v_0, v_1, \dots, v_k$ unde (v_i, v_{i+1}) sunt muchii.
 - **Lanț elementar**: vârfurile nu se repetă.
 - **Lanț simplu**: muchiile nu se repetă.
- **Drum** (în graf orientat): o succesiune de vârfuri $v_0, v_1, \dots, v_k$ unde (v_i, v_{i+1}) sunt arce orientate.
 - **Drum elementar**: vârfurile nu se repetă.
 - **Drum simplu**: arcele nu se repetă.
- **Ciclu**: un lanț simplu în care primul și ultimul vârf coincid ($v_0 = v_k$).
 - **Ciclu elementar**: toate vârfurile (cu excepția primului și ultimului) sunt distincte.
- **Circuit**: un drum simplu în care primul și ultimul vârf coincid ($v_0 = v_k$).
 - **Circuit elementar**: toate vârfurile sunt distincte (exceptând capetele).
- **Graf parțial**: obținut din graful inițial prin păstrarea tuturor vârfurilor și eliminarea unor muchii/arce.
- **Subgraf**: obținut din graful inițial prin păstrarea unei submulțimi de vârfuri și a tuturor muchiilor/arcelor care au ambele capete în această submulțime.
- **Graf transpus** (G^T): asociat unui graf orientat G , are aceleași vârfuri, iar toate arcele sunt orientate în sens invers.

3.5.2 Tipuri speciale

Graf bipartit:

- vârfurile pot fi împărțite în două mulțimi;
- muchiile unesc doar vârfuri din mulțimi diferite.

Graf complet:

- orice două vârfuri distincte sunt adiacente.

Graf turneu:

- graf orientat obținut prin orientarea fiecărei muchii dintr-un graf complet neorientat.

3.5.3 Parcurgeri

BFS:

- parcurgere în lățime;
- folosește coadă;
- utilă pentru distanțe minime în grafuri neponderate.

DFS:

- parcurgere în adâncime;
- folosește recursivitate sau stivă;
- utilă pentru componente, cicluri, sortare topologică în contexte orientate.

3.5.4 Conexitate și tare conexitate

- **Conexitate (graf neorientat)**: Un graf este conex dacă între oricare două vârfuri distincte există cel puțin un lanț. Componentele conexe sunt subgrafuri conexe maximale. Determinarea lor se realizează prin parcurgeri repetate BFS/DFS cu timp total $O(V + E)$.

- **Tare conexitate (graf orientat):** Un graf orientat este tare conex dacă pentru oricare două vârfuri distincte u, v există atât drum de la u la v , cât și de la v la u . Componentele tare conexe (CTC) sunt subgrafuri tare conexe maximale.

Algoritmul Kosaraju-Sharir pentru CTC (Complexitate: $O(V + E)$):

1. Se rulează o parcurgere DFS pe graful orientat G . Pe măsură ce vârfurile își finalizează vizita, sunt adăugate într-o stivă (sau listă).
2. Se construiește graful transpus G^T .
3. Se extrag vârfurile din stivă în ordine inversă a finalizării. Dacă un vârf nu a fost vizitat în a doua etapă, se lansează o parcurgere DFS din el în graful transpus G^T . Toate vârfurile vizitate în această parcurgere formează o CTC distinctă.

Algoritmul lui Tarjan pentru CTC (Complexitate: $O(V + E)$):

- Utilizează o singură parcurgere DFS, asociind fiecărui vârf u doi indici: $dfn[u]$ (ordinea de descoperire) și $low[u]$ (cel mai mic index al unui vârf accesibil din u prin arce DFS sau arce înapoi). Vârfurile sunt stocate pe o stivă în timpul parcurgerii. Când se identifică un vârf rădăcină de componentă ($dfn[u] == low[u]$), toate vârfurile aflate deasupra lui u în stivă sunt extrase și formează o CTC.

Matricea drumurilor / lanțurilor: O matrice pătratică R de dimensiune $N \times N$ unde $R[i][j] = 1$ dacă există drum/lanț între i și j , respectiv 0 în caz contrar (închidere tranzitivă). Se calculează utilizând algoritmul **Roy-Warshall**:

```
for (int k = 1; k <= n; k++)
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            if (a[i][j] == 0 && a[i][k] == 1 && a[k][j] == 1)
                a[i][j] = 1;
```

3.5.5 Grafuri ponderate

Drumuri de cost minim:

- Dijkstra: costuri nenegative, de la o sursă;
- Roy-Floyd: drumuri minime între toate perechile de vârfuri.

Arbori parțiali de cost minim:

- Kruskal: sortează muchiile și unește componente;
- Prim: extinde arborele cu muchia minimă care ajunge la un vârf nou.

3.5.6 Hamiltonian, eulerian și arbori

Graf hamiltonian:

- conține ciclu care trece prin toate vârfurile exact o dată.

Graf eulerian:

- conține ciclu care folosește toate muchiile exact o dată;
- pentru graf neorientat conex, toate gradele trebuie să fie pare.

Arbori:

- grafuri conexe fără cicluri;
- pentru n vârfuri au $n - 1$ muchii.

Checklist:

- Pot defini termenii fundamentali ai grafurilor.
- Pot alege reprezentarea potrivită.

- Pot aplica BFS și DFS.
- Pot determina conexitate și componente.
- Pot explica Dijkstra, Roy-Floyd, Kruskal și Prim.
- Pot diferenția grafurile hamiltoniene și euleriene.

3.6 Baze de date

Practic: Capitolul intră la proba practică pentru operațiile marcate în programă.

3.6.1 Concepte de bază

Bază de date: colecție organizată de date, structurată pentru stocare, căutare și actualizare. SGBD: sistem software care permite definirea, manipularea, interogarea și administrarea bazelor de date.

Modele clasice de organizare:

- **Modelul relațional:** Datele sunt organizate în tabele (relații) formate din rânduri (înregistrări/tupluri) și coloane (atribute). Legăturile între date se stabilesc prin valori comune stocate în chei primare și chei străine. Este modelul standard în prezent.
- **Modelul ierarhic:** Datele sunt structurate sub formă de arbore. Relațiile dintre înregistrări sunt ierarhice (părinte-copil), de tip 1 : M (un părinte poate avea mai mulți fii, dar un fiu are un singur părinte). Exemplu istoric: sistemul IMS de la IBM.
- **Modelul rețea:** Permite o structură generală de graf, în care o înregistrare copil poate avea mai mulți părinți, reprezentând direct relații de tip $M : N$ (mulți la mulți). Relațiile sunt implementate prin pointeri fizici între înregistrări. Exemplu istoric: specificațiile CODASYL.

3.6.2 Model conceptual

Elemente:

- entități;
- instanțe;
- atribute;
- identificator unic;
- relații între entități;
- cardinalități;
- normalizare.

Pași:

1. identifici entitățile;
2. stabilești atributele;
3. alegi identificatorii;
4. determini relațiile;
5. verifici redundanțele;
6. normalizezi.

3.6.3 Model relațional/fizic

Concepte:

- tabel;
- înregistrare;
- câmp;
- tip de date;
- cheie primară;

- cheie străină;
- constrângeri;
- integritate referențială.

Regulă: relațiile multe-la-multe se transformă prin tabel intermediar.

3.6.4 Normalizare

1FN:

- valori atomice;
- fără grupuri repetitive.

2FN:

- este în 1FN;
- fiecare atribut necheie depinde de întreaga cheie primară.

3FN:

- este în 2FN;
- nu există dependențe tranzitive între atribute necheie.

La examen: La examen, explică normalizarea prin dependențe funcționale și prin reducerea redundanței/anomaliilor de inserare, ștergere și modificare.

3.6.5 SQL

Operații de definire a datelor (DDL) și modificarea structurii:

- **Crearea tabelelor cu constrângeri:**

```
CREATE TABLE Clase (  
    id_clasa INT PRIMARY KEY,  
    nume_clasa VARCHAR(10) NOT NULL UNIQUE  
);
```

```
CREATE TABLE Elevi (  
    id_elev INT PRIMARY KEY,  
    nume VARCHAR(50) NOT NULL,  
    id_clasa INT,  
    media DECIMAL(4,2) CHECK (media >= 1.0 AND media <= 10.0),  
    FOREIGN KEY (id_clasa) REFERENCES Clase(id_clasa) ON DELETE SET NULL  
);
```

- **Modificarea structurii (ALTER):**

```
ALTER TABLE Elevi ADD data_nasterii DATE;  
ALTER TABLE Elevi DROP COLUMN data_nasterii;
```

- **Ștergerea tabelelor (DROP):**

```
DROP TABLE Elevi;
```

Operații de manipulare a datelor (DML):

- **SELECT:**

```
SELECT nume, media FROM Elevi WHERE media >= 8.5 ORDER BY media DESC;
```

- **INSERT:**

```
INSERT INTO Elevi(id_elev, nume, id_clasa, media) VALUES (101, 'Popescu Ana', 1,  
9.75);
```

- **UPDATE:**

Elevi SET media = 9.80 WHERE id_elev = 101;

- **DELETE:**

DELETE FROM Elevi WHERE id_elev = 101;

Interogări complexe:

- JOIN;
- GROUP BY;
- HAVING;
- funcții agregat: COUNT, SUM, AVG, MIN, MAX;
- subinterogări;
- ordonare cu ORDER BY.

Checklist:

- Pot defini baza de date și SGBD-ul.
- Pot proiecta model conceptual și model fizic.
- Pot identifica chei primare și străine.
- Pot normaliza până la 3FN.
- Pot scrie interogări SQL simple și complexe.

3.7 Arhitectura generală a sistemelor de calcul

3.7.1 Sistem de calcul

Un sistem de calcul include hardware, software și utilizatori/proceduri. Componentele interacționează pentru prelucrarea automată a datelor.

Tipuri:

- calculatoare personale;
- servere;
- sisteme mobile;
- sisteme integrate;
- sisteme distribuite.

3.7.2 Calculator personal

Unitatea centrală:

- procesor;
- memorie internă;
- placă de bază;
- magistrale;
- interfețe.

Procesor:

- unitate de comandă;
- unitate aritmetico-logică;
- registre;
- cache.

Memorie internă:

- RAM;
- ROM;

- cache.

Memorie externă:

- HDD;
- SSD;
- medii optice;
- memorii flash.

Periferece:

- intrare;
- ieșire;
- intrare-ieșire.

3.7.3 Ergonomie și siguranță

Trebuie cunoscute:

- poziția corectă la calculator;
- iluminarea;
- distanța față de monitor;
- pauzele;
- securitatea electrică;
- protecția datelor;
- sănătatea utilizatorului.

Checklist:

- Pot descrie componentele unui calculator.
- Pot explica rolul procesorului și memoriei.
- Pot clasifica perifericele.
- Pot enumera reguli de ergonomie și siguranță.

3.8 Sisteme de operare

Practic: Elementele marcate cu (*) se pot evalua și practic.

3.8.1 Concepte

Sistemul de operare gestionează resursele hardware și oferă servicii programelor și utilizatorilor.

Funcții:

- gestionarea proceselor;
- gestionarea memoriei;
- gestionarea fișierelor;
- gestionarea dispozitivelor;
- interfață cu utilizatorul;
- securitate.

Tipuri:

- monoutilizator/multiutilizator;
- monotasking/multitasking;
- desktop;
- server;
- mobile;

- timp real.

3.8.2 Interfață și accesorii

Elemente:

- desktop;
- ferestre;
- meniuri;
- pictograme;
- bare de instrumente;
- panou de control/setări.

Aplicații:

- editoare grafice;
- aplicații de animație și modelare 3D;
- aplicații audio-video;
- utilitare.

3.8.3 Sistem de fișiere

Operații:

- creare;
- copiere;
- mutare;
- redenumire;
- ștergere;
- căutare;
- arhivare/dezarhivare;
- organizare în foldere.

Concepte:

- fișier;
- folder/director;
- extensie;
- cale absolută;
- cale relativă;
- drepturi de acces.

3.8.4 Securitate, viruși, licențe

Teme:

- malware;
- viruși;
- antivirus;
- firewall;
- backup;
- autentificare;
- drepturi de autor;
- licențe software;
- utilizarea legală a conținuturilor digitale.

Checklist:

- Pot defini sistemul de operare și funcțiile lui.
- Pot explica organizarea logică a datelor.
- Pot descrie operații cu fișiere și foldere.
- Pot explica securitatea datelor și licențele software.

3.9 Aplicații de birotică

Practic: Capitolul intră la proba practică.

3.9.1 Procesoare de text

Competențe:

- creare și salvare document;
- formatare caractere și paragrafe;
- stiluri;
- liste;
- tabele;
- inserare imagini;
- antet/subsol;
- numerotare pagini;
- cuprins;
- verificare ortografică;
- export PDF.

La examen practic, răspunsul bun respectă exact cerințele de formatare: font, dimensiune, aliniere, spațiere, margini, tabele, imagini.

3.9.2 Calcul tabelar

Competențe:

- celule, rânduri, coloane;
- formatare;
- formule;
- funcții;
- sortare;
- filtrare;
- diagrame;
- referințe relative și absolute.

Funcții uzuale:

- SUM;
- AVERAGE;
- MIN;
- MAX;
- COUNT;
- IF;
- COUNTIF;
- SUMIF;
- VLOOKUP/căutări echivalente.

3.9.3 Presentări

Competențe:

- creare diapozitive;
- alegerea temei;
- inserare text, imagini, tabele, diagrame;
- tranziții și animații;
- reguli de lizibilitate;
- prezentare coerentă.

Checklist:

- Pot realiza documente text formatate corect.
- Pot folosi formule și funcții într-un editor tabelar.
- Pot crea diagrame și interpreta date.
- Pot realiza o prezentare coerentă și lizibilă.

3.10 Rețele de calculatoare

Practic: Internetul, aplicațiile colaborative și HTML sunt zone de probă practică.

3.10.1 Concepte

Rețea de calculatoare: ansamblu de echipamente interconectate pentru comunicare și partajare de resurse.

Tipuri:

- PAN;
- LAN;
- MAN;
- WAN;
- Internet.

Funcții:

- comunicare;
- partajare fișiere;
- partajare imprimante;
- acces la servicii;
- colaborare.

Protocoale:

- TCP/IP;
- HTTP/HTTPS;
- FTP;
- SMTP;
- POP3;
- IMAP;
- DNS.

3.10.2 Internet

Teme:

- acces la Internet;
- adresare IP;
- nume de domeniu;
- servicii: web, e-mail, transfer fișiere, mesagerie, cloud;
- aplicații colaborative;

- siguranță online.

3.10.3 HTML

HTML este limbaj de marcare pentru structurarea paginilor web. Se cere cunoașterea elementelor de bază:

- structură document;
- titluri;
- paragrafe;
- liste;
- imagini;
- linkuri;
- tabele;
- formulare simple.

Model:

```
<!doctype html>
<html lang="ro">
<head>
  <meta charset="utf-8">
  <title>Pagina mea</title>
</head>
<body>
  <h1>Titlu</h1>
  <p>Text introductiv.</p>
  <a href="https://example.com">Legătură</a>
</body>
</html>
```

Checklist:

- Pot clasifica rețelele și protocoalele.
- Pot explica serviciile Internet.
- Pot descrie e-mailul și protocoalele asociate.
- Pot scrie o pagină HTML simplă.

4. Metodica predării disciplinelor informatice

4.1 Proiectarea, organizarea și desfășurarea activității didactice

4.1.1 Curriculum național

Concepte și documente reglatoare:

- **Curriculum național:** Sistemul coerent de planuri-cadru, programe școlare și metodologii de predare-învățare-evaluare aplicat în sistemul național.
- **Arie curriculară:** O grupare interdisciplinară de discipline școlare (de exemplu: „Matematică și Științe ale naturii”, din care fac parte Informatica și TIC).
- **Plan-cadru:** Documentul oficial care stabilește disciplinele studiate pe ani de studiu și numărul minim și maxim de ore pe săptămână.
- **Plan de învățământ:** Detalierea planului-cadru la nivelul fiecărei școli și profil.
- **Programă școlară:** Documentul care stabilește competențele generale și specifice, conținuturile științifice și sugestiile metodologice pentru o anumită disciplină.
- **Manual școlar:** Instrument de lucru concret pentru elevi, detaliind conținuturile programei.

- **Auxiliar didactic:** Ghiduri, culegeri, softuri educaționale aprobate care sprijină procesul didactic.
- **Curriculum diferențiat:** Adaptarea conținuturilor pentru filiera/profilul clasei (ex: Informatică intensiv vs. neintensiv).
- **Curriculum adaptat:** Modificarea curriculumului pentru elevii cu cerințe educaționale speciale (CES).
- **Curriculum la decizia școlii (CDȘ) / în dezvoltare locală (CDL):** Orele opționale oferite de școală în funcție de resursele locale și interesele elevilor.
- **Modularitate:** Organizarea instruirii în unități de sine stătătoare (module) care pot fi parcurse independent sau combinate flexibil, specifică învățământului tehnic și profesional, permițând individualizarea ritmului de învățare.

Răspuns de examen:

- definește termenul;
- precizează rolul în proiectarea didactică;
- dă exemplu pentru Informatică/TIC.

4.1.2 Documente de proiectare

Planificare calendaristică:

- distribuie conținuturile pe intervale de timp;
- corelează competențe, conținuturi, activități și evaluare;
- ține cont de structura anului școlar.

Proiectul unității de învățare:

- competențe specifice;
- conținuturi;
- activități de învățare;
- resurse;
- evaluare;
- timp alocat.

Proiectul de lecție:

- date generale;
- competențe/obiective;
- strategii;
- resurse;
- scenariu didactic;
- evaluare.

4.1.3 Demersuri didactice

- **Intradisciplinar:** Stabilirea de legături strânse între capitolele aceleiași discipline (ex: corelarea structurilor de date dinamice cu programarea orientată pe obiecte în C++).
- **Multidisciplinar:** Studiarea unei teme comune din perspectiva mai multor discipline distincte în paralel, păstrându-și identitatea proprie (ex: analiza datelor climatice la Geografie și realizarea graficelor în Excel la TIC).
- **Interdisciplinar:** Integrarea activă a metodelor, conceptelor și termenilor din mai multe discipline pentru a rezolva o problemă comună (ex: utilizarea funcțiilor matematice din trigonometrie pentru a scrie cod de desenare grafică în Informatică).

- **Transdisciplinar:** Demers profund integrat ce transcende complet barierele disciplinelor clasice, concentrându-se pe rezolvarea unor probleme de viață reală (ex: dezvoltarea unui proiect antreprenorial digital de către elevi, integrând programare, economie, marketing și design).

Checklist:

- Pot explica documentele curriculare.
- Pot construi o planificare sau o unitate de învățare.
- Pot diferenția intra-, inter- și transdisciplinaritatea.
- Pot formula o ofertă pentru CDȘ/CDL.

4.2 Strategii didactice

4.2.1 Tipuri de strategii

Strategia didactică este ansamblul metodelor, mijloacelor, formelor de organizare și tipurilor de activități prin care se urmărește atingerea competențelor.

Strategii:

- expozitive;
- euristice;
- algoritmice;
- problematizante;
- practice/aplicative;
- colaborative;
- asistate de calculator.

4.2.2 Lecția

Tipuri de lecții:

- dobândire de cunoștințe;
- formare de priceperi și deprinderi;
- recapitulare;
- sistematizare;
- evaluare;
- mixtă.

Momente:

- organizarea clasei;
- captarea atenției;
- anunțarea temei și obiectivelor;
- reactualizarea cunoștințelor;
- predarea/învățarea;
- fixarea;
- evaluarea;
- tema pentru acasă.

4.2.3 Metode didactice

Metode cerute în programă:

- observarea;
- exercițiul;
- modelarea;

- problematizarea;
- demonstrarea;
- învățarea prin descoperire;
- expunerea sistematică;
- conversația;
- aplicațiile practice;
- instruirea programată;
- învățarea asistată de calculator;
- metode activ-participative centrate pe elev.

Exemplu de aplicare - învățarea prin descoperire la algoritmi:

- profesorul propune o secvență de valori;
- elevii observă un tipar;
- formulează ipoteza;
- o testează prin pseudocod;
- generalizează algoritmul;
- îl implementează.

4.2.4 Forme de organizare

Frontal:

- eficient pentru explicații și demonstrații;
- risc de pasivitate.

Individual:

- bun pentru exersare și diferențiere;
- cere monitorizare.

Pe grupe:

- bun pentru proiecte și rezolvare colaborativă;
- cere roluri clare și criterii de evaluare.

4.2.5 Mijloace de învățământ și resurse digitale

Mijloace:

- manual;
- fișe;
- tablă;
- calculator;
- videoproiector;
- mediu de programare;
- aplicații de birotică;
- platforme e-learning;
- Internet;
- software educațional.

Integrarea mijloacelor trebuie justificată prin competența urmărită, nu doar enumerată.

Checklist:

- Pot descrie tipuri de strategii didactice.
- Pot construi momentele unei lecții.
- Pot alege metode potrivite pentru algoritmi, TIC și baze de date.

- Pot justifica folosirea resurselor digitale.

4.3 Evaluarea randamentului școlar

4.3.1 Concept și funcții

Evaluarea măsoară și apreciază nivelul de realizare a competențelor. Funcții:

- diagnostică;
- prognostică;
- formativă;
- sumativă;
- motivațională;
- de reglare.

Tipuri:

- inițială;
- continuă/formativă;
- finală/sumativă.

4.3.2 Metode și instrumente

Tradiționale:

- probe orale;
- probe scrise;
- probe practice.

Alternative/complementare:

- proiect;
- portofoliu;
- investigație;
- observare sistematică;
- autoevaluare;
- interevaluare.

Calități fundamentale ale instrumentelor de evaluare:

- **Validitate:** Capacitatea testului de a măsura exact ceea ce își propune să măsoare (de exemplu, o lucrare practică la TIC să măsoare competențele digitale, nu viteza de dactilografie).
- **Fidelitate:** Proprietatea unui test de a oferi rezultate constante și reproductibile dacă este aplicat în aceleași condiții aceleiași clase sau unor clase similare.
- **Obiectivitate:** Gradul de concordanță între notele acordate independent de mai mulți evaluatori. Este maximă la itemii obiectivi și minimă la cei subiectivi.
- **Aplicabilitate:** Ușurința cu care testul poate fi administrat, evaluat și interpretat din punct de vedere al resurselor (timp, tehnică de calcul).
- **Sensibilitate / Putere de discriminare:** Capacitatea testului de a diferenția elevii foarte pregătiți de cei cu o pregătire medie sau slabă.

4.3.3 Itemi

Un item reprezintă cel mai mic element dintr-un test de evaluare, constând în enunț și răspuns așteptat.

1. Itemi obiectivi (alegere duală, alegere multiplă, asociere):

- **Reguli de proiectare:** Distractorii (opțiunile greșite) trebuie să fie plauzibili; enunțul trebuie să evite negațiile multiple.
- **Avantaje:** Evaluare extrem de rapidă, obiectivitate 100% (elimină subiectivismul evaluatorului), acoperire largă a materiei.
- **Dezavantaje / Limite:** Nu pot măsura abilitatea de exprimare sau pașii logici din spatele unei rezolvări; risc mare de ghicire a răspunsului corect.

2. Itemi semiobiectivi (răspuns scurt, completare, întrebări structurate):

- **Reguli de proiectare:** Spațiul lăsat liber pentru completare trebuie să permită un singur răspuns corect (fără ambiguități); întrebările structurate trebuie să aibă sub-cerințe cu dificultate gradată.
- **Avantaje:** Permit elevului să construiască răspunsul (nu doar să îl selecteze), oferă posibilitatea punctării parțiale, reduc șansa de ghicire.
- **Dezavantaje:** Cer mai mult timp pentru corectare decât cei obiectivi; pot apărea dificultăți de corectare din cauza scrisului sau sinonimelor.

3. Itemi subiectivi / cu răspuns deschis (rezolvare de probleme, eseuri structurate, proiecte, referate):

- **Reguli de proiectare:** Formularea cerinței trebuie să delimiteze clar pașii așteptați; este obligatoriu un barem detaliat de notare pe secvențe de cod sau etape de lucru.
- **Avantaje:** Măsoară abilitățile de analiză, sinteză, argumentare și rezolvare de probleme complexe (de exemplu, scrierea unui algoritm eficient de la zero).
- **Dezavantaje:** Obiectivitate redusă (notare influențată de corector), corectare extrem de lentă, număr mic de teme testate în unitatea de timp.

Reguli de proiectare generală a itemilor la Informatică:

- Enunțul să fie neambiguu și precis definit (ex: precizarea dacă un tablou are indici de la 0 la $n-1$ sau 1 la n).
- Barem clar bazat pe componente (ex: punctaj distinct pentru citire, logica algoritmului, declararea variabilelor, sintaxă și tratarea cazurilor limită).

4.3.4 Notare și erori de evaluare

Factori perturbatori:

- efectul halo;
- efectul de contrast;
- eroarea de severitate/indulgență;
- tendința centrală;
- stereotipii;
- influența ordinii lucrărilor.

Minimizare:

- barem clar;
- criterii explicite;
- evaluare anonimă unde se poate;
- compararea cu descriptorii de performanță;
- feedback constructiv.

Checklist:

- Pot explica funcțiile evaluării.
- Pot clasifica metodele de evaluare.

- Pot proiecta itemi și bareme.
- Pot explica erorile de notare și soluțiile de diminuare.

5. Proba practică

5.1 Domenii de pregătit

Conform notei din programă, proba practică se susține din capitolele:

- 2, elementele marcate (*): limbaje de programare;
- 6, elementele marcate (*): baze de date;
- 8, elementele marcate (*): sisteme de operare;
- 9: aplicații de birotică;
- 10, elementele marcate (*): Internet și HTML.

5.2 Programare

Exersează:

- citire/scriere de la tastatură;
- citire/scriere din fișiere text;
- tablouri și matrice;
- șiruri de caractere;
- subprograme;
- recursivitate;
- algoritmi elementari;
- sortare și căutare.

Checklist practic:

- Pot crea rapid un program complet în C++ sau Pascal.
- Pot folosi fișiere text fără erori de deschidere/închidere.
- Pot testa cazuri limită: 0, 1, șir vid, valori negative, duplicate.
- Pot explica soluția și complexitatea.

5.3 Baze de date

Exersează:

- creare tabele;
- alegere tipuri de date;
- chei primare și străine;
- introducere date;
- modificare date;
- ștergere date;
- interogări simple;
- interogări cu join, agregare și grupare.

Checklist practic:

- Pot transforma un enunț în tabele relaționale.
- Pot scrie CREATE TABLE cu constrângeri.
- Pot scrie SELECT cu JOIN, GROUP BY, HAVING.
- Pot interpreta rezultatul unei interogări.

5.4 Sisteme de operare și fișiere

Exersează:

- creare foldere;
- copiere/mutare/redenumire fișiere;
- arhivare/dezarhivare;
- căutare fișiere;
- setări de bază;
- aplicații grafice/audio-video dacă sunt cerute;
- respectarea drepturilor de autor și licențelor.

5.5 Birotică

Text:

- formatare pagină;
- stiluri;
- tabele;
- imagini;
- antet/subsol;
- cuprins;
- export.

Calcul tabelar:

- formule;
- funcții;
- sortare/filtrare;
- diagrame.

Prezentări:

- structură;
- design coerent;
- imagini;
- animații/tranziții moderate.

5.6 HTML și Internet

Exersează:

- structură HTML;
- titluri/paragrafe;
- liste;
- imagini;
- linkuri;
- tabele;
- formulare simple;
- salvarea și deschiderea paginii în browser.

6. Plan de studiu

6.1 Etapa 1: Fundamentare algoritmică

Durată recomandată: 2-3 săptămâni.

Teme:

- algoritmi elementari;
- sortări;

- căutări;
- complexitate;
- fișiere;
- subprograme.

Rezultat așteptat:

- poți rezolva probleme scurte fără să cauți model;
- poți scrie aceeași idee în C++ și Pascal;
- poți explica de ce algoritmul este corect.

6.2 Etapa 2: Metode avansate și structuri

Durată recomandată: 3-4 săptămâni.

Teme:

- greedy;
- backtracking;
- divide et impera;
- programare dinamică;
- alocare dinamică;
- liste, stive, cozi, arbori.

Rezultat așteptat:

- identifici metoda potrivită;
- scrii schelete reutilizabile;
- analizezi complexitatea.

6.3 Etapa 3: Grafuri

Durată recomandată: 2-3 săptămâni.

Teme:

- reprezentări;
- BFS/DFS;
- conexitate;
- drumuri minime;
- arbori parțiali minimi;
- grafuri hamiltoniene și euleriene.

Rezultat așteptat:

- poți rezolva cerințe teoretice și aplicative pe grafuri;
- poți face urme de algoritm în tabel.

6.4 Etapa 4: Baze de date și TIC

Durată recomandată: 3 săptămâni.

Teme:

- model conceptual;
- model relațional;
- normalizare;
- SQL;
- arhitectura calculatorului;
- sisteme de operare;

- rețele;
- birotică;
- HTML.

Rezultat așteptat:

- poți rezolva subiecte de baze de date;
- poți susține proba practică pentru TIC;
- poți explica noțiuni de sistem și rețea.

6.5 Etapa 5: Metodică și evaluare

Durată recomandată: 3-4 săptămâni.

Teme:

- curriculum;
- planificare;
- proiect de lecție;
- strategii didactice;
- metode;
- evaluare;
- itemi;
- bareme;
- erori de notare.

Rezultat așteptat:

- poți construi răspunsuri complete la Subiectul III;
- poți justifica metodele didactice;
- poți formula instrumente de evaluare.

6.6 Etapa 6: Recapitulare pe subiecte

Durată recomandată: 2-4 săptămâni.

Folosește:

- rezolvările anuale din repository;
- statistici_subiecte.typ;
- modelele 2024-2026;
- proba practică 2025-2026.

Rutina recomandată:

1. rezolvă subiectul cronometrat;
2. compară cu baremul;
3. completează lacunele în ghid;
4. refă subiectele cu teme recurente;
5. simulează examenul complet.

7. Matrice de verificare finală

7.1 Algoritmă și programare

- Noțiunea de algoritm și caracteristici.
- Structuri fundamentale și programare structurată.
- Reprezentări: pseudocod, scheme, blocuri grafice.

- Algoritmi elementari pe numere și secvențe.
- Sortări: bule, inserție, selecție, numărare.
- Interclasare.
- Căutare secvențială și binară.
- Complexitate timp/spațiu.
- C/C++/Pascal: sintaxă, tipuri, instrucțiuni.
- Tablouri, matrice, înregistrări, șiruri.
- Fișiere text.
- Subprograme și recursivitate.
- Programare orientată pe obiecte.

7.2 Metode și structuri

- Greedy.
- Backtracking.
- Divide et impera.
- Programare dinamică.
- Generări combinatoriale.
- Alocare dinamică.
- Liste, stive, cozi.
- Arbori și arbori binari.

7.3 Grafuri

- Definiții și terminologie.
- Reprezentări.
- Grafuri speciale.
- BFS și DFS.
- Conexitate și tare conexitate.
- Dijkstra și Roy-Floyd.
- Hamiltonian și eulerian.
- Kruskal și Prim.

7.4 Baze de date și TIC

- Concepte BD și SGBD.
- Model conceptual.
- Model fizic relațional.
- Normalizare.
- SQL.
- Arhitectura sistemelor de calcul.
- Sisteme de operare.
- Birotică.
- Rețele, Internet, HTML.

7.5 Metodică

- Curriculum și documente școlare.
- Planificare calendaristică.
- Unitate de învățare.
- Proiect de lecție.
- Strategii și metode didactice.

- Forme de organizare.
- Mijloace de învățământ.
- Software educațional și e-learning.
- Evaluare, itemi, bareme.
- Notare și erori de evaluare.

7.6 Matrice explicită de acoperire a programei

Această matrice leagă direct structura oficială a programei de secțiunile din ghid, pentru verificarea finală a parcurgerii.

Componentă din programa oficială	Acoperire în ghid	Status
Notă de prezentare și competențe ale cadrului didactic	Secțiunile 1 și 2 explică scopul, competențele profesionale, competențele didactice și modul de folosire a ghidului.	Complet
Tematică științifică: algoritmi, limbaje, metode, structuri, grafuri, baze de date, arhitectură, sisteme, birotică, rețele	Secțiunea 3 acoperă toate cele 10 capitole științifice, cu definiții, exemple, cod, operații specifice, checklisturi și capcane de examen.	Complet
Proba practică	Secțiunea 5 grupează explicit capitolele marcate cu (*): programare, baze de date, sisteme de operare, birotică, Internet și HTML.	Complet
Metodica predării disciplinelor informatice	Secțiunea 4 acoperă curriculumul, proiectarea didactică, strategiile, metodele, formele de organizare, mijloacele și evaluarea.	Complet
Bibliografie științifică și metodică	Secțiunea 8 indică folosirea manualelor, documentelor reglatoare, programelor școlare și bibliografiei metodice/științifice în mod selectiv.	Complet
Pregătire integrată pentru examen	Secțiunile 6, 7, 9 și 10 oferă planul de studiu, matricea de verificare, reguli pentru răspunsuri de punctaj maxim și legătura cu rezolvările anuale.	Complet

8. Bibliografie orientativă după programă

8.1 Tematica științifică

Sunt obligatorii manualele școlare valabile pentru disciplinele informatice de gimnaziu și liceu, precum și programele pentru examenele naționale. Pentru aprofundare:

- manuale și culegeri de C/C++ și Pascal pentru liceu;
- introduceri în algoritmi și structuri de date;
- lucrări de grafuri și combinatorică;

- lucrări de baze de date și SQL;
- materiale despre arhitectura calculatoarelor și rețele.

Titlurile din bibliografia oficială trebuie folosite selectiv, în funcție de temă. Pentru pregătirea rapidă, prioritizează manualele școlare, subiectele oficiale și rezolvările complete.

8.2 Metodica predării

Sunt obligatorii documentele școlare reglatoare valabile în anul școlar al concursului:

- planuri-cadru;
- programe școlare;
- programe pentru examene și evaluări naționale;
- metodologii aplicabile.

Pentru aprofundare:

- didactica predării informaticii;
- pedagogie generală;
- teoria și metodologia instruirii;
- teoria și metodologia evaluării;
- strategii didactice interactive;
- evaluare asistată de calculator;
- e-learning.

9. Reguli pentru răspunsuri de punctaj maxim

9.1 La probleme de programare

Un răspuns complet include:

- explicarea ideii;
- algoritm/pseudocod sau cod;
- tratarea cazurilor limită;
- complexitate;
- justificarea corectitudinii, cel puțin prin argument logic.

Evită:

- cod fără explicație;
- variabile neinițializate;
- citire greșită din fișier;
- complexitate neverificată;
- ignorarea restricțiilor.

9.2 La baze de date

Un răspuns complet include:

- entități și attribute;
- chei primare;
- chei străine;
- relații și cardinalități;
- normalizare;
- SQL corect sintactic și semantic.

Evită:

- tabel unic cu date redundante;

- lipsa cheilor;
- relații multe-la-multe nerezolvate;
- GROUP BY greșit;
- agregări folosite fără criteriu.

9.3 La metodică

Un răspuns complet include:

- competența vizată;
- conținutul;
- metoda;
- forma de organizare;
- resursele;
- activitatea profesorului;
- activitatea elevilor;
- evaluarea;
- justificarea alegerilor.

Evită:

- enumerări fără legătură cu lecția;
- metode alese generic;
- obiective neobservabile;
- evaluare fără criterii;
- lipsa adaptării la vârsta elevilor.

10. Legătura cu materialele din repository

Pentru consolidare:

- folosește rezolvările anuale 2011-2026 pentru exerciții concrete;
- folosește statistici_subiecte.typ pentru teme recurente;
- folosește rezolvările de probă practică pentru partea aplicativă;
- revino la acest ghid pentru a verifica dacă fiecare subiect rezolvat acoperă un punct din programă.

Ordine practică:

1. citește capitolul din ghid;
2. rezolvă un subiect oficial relevant;
3. compară cu rezolvarea din repository;
4. notează tiparul cerinței;
5. refă fără ajutor după 3-5 zile.

11. Comandă de compilare

Dacă Typst este instalat local, PDF-ul se generează cu:

```
typst compile "Informatică și tehnologia informației/  
ghid_programa_titularizare_informatica_tic.typ" "Informatică și tehnologia  
informației/ghid_programa_titularizare_informatica_tic.pdf"
```

Pentru lucru iterativ:

```
typst watch "Informatică și tehnologia informației/  
ghid_programa_titularizare_informatica_tic.typ"
```